

Trabajo Practico Integrador - Diagnostico y Monitoreo Multiplataforma

Arquitectura y Sistemas Operativos

Alumnos: Ricardo Oliva y Facundo Gómez
25-06-2026

Diagnóstico de Red y Monitoreo de Recursos Multiplataforma

Tema: Virtualización

Alumnos: Ricardo Oliva y Facundo Gómez

Materia: Arquitectura y Sistemas Operativos

Profesor: Nicolas Llanea

Fecha de Entrega: 25 de junio de 2026

Índice

1. Introducción
2. Marco Teórico
3. Caso Práctico y Problemática
4. Metodología Utilizada
5. Resultados Obtenidos
6. Conclusiones
7. Bibliografía

1. Introducción

En el ámbito de la administración de sistemas, la verificación del estado de la red y el monitoreo del consumo de recursos de hardware son tareas fundamentales para garantizar la disponibilidad y el correcto rendimiento de los servidores y terminales de trabajo.

Históricamente, los administradores han dependido de herramientas nativas e individuales de cada sistema operativo, lo que fragmenta el flujo de trabajo cuando se gestionan entornos mixtos.

Este trabajo práctico aborda el desarrollo e implementación de una solución unificada mediante un script ejecutable en lenguaje Python. La herramienta está diseñada para funcionar de manera nativa tanto en entornos Windows como en sistemas basados en Linux, específicamente testado bajo la distribución Ubuntu Server ejecutada en un entorno virtualizado con Oracle VirtualBox. De esta manera, se unifica el diagnóstico básico de conectividad y la inspección de carga del procesador y memoria RAM en una sola interfaz de consola de comandos.

2. Marco Teórico

Para comprender el funcionamiento interno del script de diagnóstico desarrollado, es necesario definir los pilares tecnológicos sobre los cuales se fundamenta:

- **Portabilidad Multiplataforma:** Capacidad del software para ejecutarse en diferentes arquitecturas y sistemas operativos sin requerir modificaciones en su código fuente base. En Python, esto se gestiona abstrayendo las llamadas al sistema mediante módulos estándar como `os`.
- **Interrupción y Control de Procesos (Módulo `Subprocess`):** Permite a Python invocar directamente binarios de la consola del sistema anfitrión (como `ping`, `ipconfig` o `free`), capturar sus canales de salida estándar (`stdout`) y de errores (`stderr`), y gestionar tiempos de espera críticos (`timeouts`) para evitar bloqueos indefinidos del hilo principal.
- **Protocolo ICMP (Ping):** Herramienta de diagnóstico que utiliza paquetes de solicitud y respuesta de eco para determinar si un host remoto está activo y accesible a través de la infraestructura de red.
- **Virtualización de Tipo 2 (Hosted):** Ejecución de un sistema operativo completo (invitado) dentro de un software que corre sobre otro sistema operativo (anfitrión). Para este proyecto, el uso de VirtualBox permite emular el comportamiento del script en servidores de producción Linux sin comprometer la máquina principal de desarrollo.

Métrica / Comando	Implementación Windows	Implementación Linux (Ubuntu)
Estructura de Red	ipconfig (Puerta de enlace)	ip route (Default via)
Diagnóstico ICMP	ping -n 1 [Host]	ping -c 1 [Host]
Métricas de CPU	wmic cpu get LoadPercentage	top -b -n 1 (Primeras líneas)
Métricas de RAM	wmic OS get FreePhysicalMemory...	free -m

3. Caso Práctico y Problemática

Problemática Encontrada: La administración descentralizada de estaciones de trabajo obliga al personal de soporte técnico a recordar y adaptar sintaxis de comandos que varían críticamente entre plataformas. Por ejemplo, el parámetro para definir la cantidad de paquetes en un ping cambia de `-n` en Windows a `-c` en Linux. Asimismo, los tiempos de respuesta fallidos

en comandos secuenciales provocan congelamientos prolongados en terminales automatizadas si no se parametrizan correctamente los desbordamientos de tiempo.

Solución Diseñada: Se estructuró un programa en Python modularizado en funciones específicas:

1. **detectar_so():** Interroga al intérprete para clasificar el entorno de ejecución en entorno NT (Windows) o POSIX (Linux).
2. **optener_ip_router():** Automatiza el parseo de cadenas de texto para extraer dinámicamente la dirección IP de la puerta de enlace predeterminada de la red local actual, aislando fallos mediante un bloque try-except que devuelve un fallback estándar (192.168.1.1) si ocurren errores de permisos.
3. **diagnostico_red():** Valida de manera jerárquica la conectividad local (Router), la resolución externa (DNS de Google y Cloudflare) y la capa de aplicación web (GitHub) usando la función segura de ping con timeout estricto de 3 segundos.
4. **sensor_recursos():** Captura y formatea de manera limpia el uso de hardware en tiempo real sin requerir librerías pesadas de terceros como psutil, utilizando únicamente recursos nativos del sistema.

4. Metodología Utilizada

El proceso de desarrollo, despliegue y testeo del software se dividió en las siguientes fases metodológicas:

- **Fase 1: Codificación e Inyección de Robustez:** Escritura del código en un entorno de desarrollo integrado, implementando el manejo explícito de excepciones (ValueError, CalledProcessError) para blindar el menú interactivo contra entradas nulas o caracteres inválidos.
- **Fase 2: Despliegue en Entorno Anfitrión (Windows):** Validación de la captura de datos a través de la herramienta instrumental de administración de Windows (WMIC) y testeo de la interfaz cromática mediante códigos de escape ANSI en la consola.
- **Fase 3: Configuración del Entorno Virtual (Linux):** Despliegue de una máquina virtual en Oracle VirtualBox configurada con Ubuntu Server. Para asegurar que el módulo de diagnóstico de red pudiese validar el entorno real, se configuró la interfaz de red de la máquina virtual en **Modo Bridge (Adaptador Puente)**. Esto otorgó al sistema operativo invitado una dirección IP propia dentro del mismo rango del router físico.
- **Fase 4: Ejecución e Interoperabilidad:** Transferencia del script a la máquina virtual mediante SSH/SCP y ejecución del programa en la terminal Linux para comprobar la correcta bifurcación lógica orientada a comandos como ip route y free -m.

5. Resultados Obtenidos

La herramienta multiplataforma demostró un comportamiento óptimo y predecible en ambos entornos evaluados, logrando los siguientes hitos:

- **Aislamiento Eficiente de Fallas de Red:** Durante las pruebas controladas (desconexión del cable de red), el script reportó instantáneamente fallas individuales en el enrutador local y servidores externos en menos de 3 segundos por objetivo, sin interrumpir el flujo del menú principal gracias al control de desbordamiento por tiempo (timeout).
- **Adaptabilidad en Entorno Virtual:** En la máquina virtual con Ubuntu Server ejecutada en VirtualBox, el programa detectó con precisión el gateway virtual asignado por el DHCP del router doméstico gracias al modo bridge, imprimiendo el estado de la memoria RAM expresado correctamente en Megabytes (MB).
- **Salida Limpia y Segura:** El control de interrupciones de teclado (KeyboardInterrupt / Control+C) interceptó las señales de terminación abrupta, impidiendo la visualización de trazas de error (tracebacks) complejas y asegurando un cierre controlado del proceso en consola.

6. Conclusiones

La unificación de comandos administrativos a través de lenguajes de alto nivel como Python reduce ostensiblemente la fricción operativa en la gestión de infraestructuras tecnológicas heterogéneas. Al abstraer la complejidad subyacente de las llamadas de bajo nivel del sistema operativo, se crea un entorno de diagnóstico homogéneo, veloz y altamente escalable. Adicionalmente, el uso de hipervisores de tipo 2 como VirtualBox consolida su valor académico y profesional, permitiendo validar la lógica multiplataforma de scripts de automatización dentro de una misma estación de trabajo física, mitigando riesgos operativos y optimizando el tiempo de desarrollo de soluciones de software de infraestructura.

7. Bibliografía

- Python Software Foundation. (2026). *The Python Standard Library - Subprocess management*. Recuperado de <https://docs.python.org/3/library/subprocess.html>
- Oracle. (2025). *Oracle VM VirtualBox User Manual - Virtual Networking*. Recuperado de <https://www.virtualbox.org/manual/>
- Canonical Ltd. (2025). *Ubuntu Server Guide - Network Configuration*. Recuperado de <https://ubuntu.com/server/docs>